

INTERNATIONAL UNIVERSITY OF SARAJEVO
INTERNACIONALNI UNIVERZITET U SARAJEVU

ELEVATOR- PROJECT REPORT

Prepared by: Dženneta Mujagić, Dina Babačić

Supervisor: Dr. Tarik Namas, Associate Professor

Date: January, 2026

Contents

Rethinking an Everyday Necessity 3
Theoretical Introduction 3
Program Introduction 4
System Overview 4
System Architecture 4
Graphical User Interface Design 6
Elevator Status Panel 6
Cabin Interface Panel 6
Control and Monitoring Panel 7
Emergency Control Panel 7
Elevator Movement Simulation 7
Request Handling and Movement Logic 7
Door System and Motion Sensor 8
Weight Control System 9
Music and Lighting System 9
Emergency Systems 10
Emergency Priority Logic 10
Energy-Saving Mode 11
Failure Recovery and Reset 11
Apparatus and Tools Used 11
QR Code Feedback System 12
Customer Feedback Integration 12
Development Process 12
Educational Value and Conclusion 13
Resources: 14

Rethinking an Everyday Necessity

This project began with a daily ritual: the hum of the university elevator carrying Dženneta and Dina from the lobby to their engineering class. But that routine sparked a critical question. With every press of the button, they were confronted by the trade-off between modern convenience and environmental cost. Were we simply using too much electricity? And more importantly, was this ethical?

Motivated by this ethical and practical challenge, they shifted from passive users to active designers. Their goal was to re-engineer the elevator's core function, developing a system that prioritizes radical energy efficiency without compromising reliability. They chose a multi-story shopping mall as their conceptual testing ground, a complex environment to prove its viability.

Safety remained non-negotiable. Informed by personal encounters with the unreliable elevators in Sarajevo's older buildings, they integrated a robust, intelligent alarm system into their design from the outset.

Theoretical Introduction

Negotiable Elevator engineering integrates the disciplines of mechanics, electronics, and software to deliver safe, efficient vertical transportation. This project focuses specifically on the software component of elevator design.

The performance of an elevator system is a defining feature of modern architecture, influencing how people interact with and move through both commercial and residential spaces. Architects and engineers rely on advanced vertical-transportation strategies to balance user comfort, safety, and energy efficiency—making Elevator Simulation Software an indispensable part of the design and testing process.

Behind what seems like a simple lift mechanism lies a sophisticated network of scheduling algorithms, concurrency management, and safety-critical systems. Modern elevator software acts as a central control interface, allowing engineers to model everything from cabin dimensions to complex operational logic using tools such as C, C++, Python, Java, and MATLAB.

While traditional elevators operate on fixed, reactive logic, today’s systems are evolving. Advances in software now enable AI-driven decision-making, moving beyond basic call-response functions toward adaptive, personalized user experiences that optimize efficiency, wait times, and energy use.

Theoretical references for deeper theory:

- Elevator-evolution-and-its-impact
- Latest-evolution-in-elevator-technology

Program Introduction

This project presents the design and simulation of a smart five-story elevator system implemented using GNU Octave (MATLAB-compatible GUI).

The objective of the project is to demonstrate how engineers design and simulate control systems with safety constraints, graphical user interfaces, and state-based logic using software modelling techniques. The system is a software simulation, meaning that physical movement, sensors, and mechanical components are represented logically through program states, delays, and GUI feedback. Such simulations are commonly used in engineering to test system behaviour before real-world implementation.

System Overview

• Number of floors: 5
• Floor labels: -1, 0, 1, 2, 3
• Maximum allowed load: 800 kg
• Door type: Automatic doors with manual override
• Operating schedule: Continuous (24/7)
• Movement type: floor by floor logical simulation
• Platform: GNU octave GUI

System Architecture

The system is implemented as a single controller program, where all control logic exists in one codebase. While the program executes all tasks internally, the responsibilities are conceptually separated to reflect how real-world elevator systems are organized. This approach allows for clear organization of functionality and easier adaptation of different operational logics and regimes.

Several core functions, which the elevator control logic manages, are:

- Simulated floor-to-floor movement is displayed through GUI status updates, showing the current floor and direction rather than using graphical animation. The system moves one floor at a time, with time delays to simulate travel duration.

- A floor request stores all requested stops. Duplicate requests are automatically ignored, and requests are cleared if an emergency state is activated.

- Door control handles both automatic and manual operations. Doors only open when the elevator is fully stopped, and a simulated motion sensor prevents doors from closing if passenger movement is detected.

- Weight monitoring checks that the elevator load does not exceed the maximum allowed value of 800 kg. If the limit is surpassed, the elevator stops, clears requests, opens the doors, and resumes operation only after the load is reduced.

- Emergency handling manages fire, flood, and earthquake events. Emergency states have absolute priority over all other operations, temporarily suspending normal logic. Fire moves the elevator to floor 0, flood moves it to floor 3 and disables the basement floor, and earthquake stops the elevator immediately until manually reset.

- Energy-saving behaviour is activated when no passengers or floor requests are present. In this mode, lights turn off, music stops, doors close, and the elevator remains idle.

- Status displays on the GUI provide continuous feedback, including the current floor, elevator direction (up, down, or idle), door state (open or closed), energy-saving mode status, and emergency alerts.

- Music and lighting systems are simulated through GUI feedback, allowing users to turn them on or off, select songs, and adjust volume, while ensuring they automatically pause during emergencies or energy-saving mode.

- Day-based operations**:** Sunday maintenance mode disables normal operations while keeping emergency systems active

- Weekend energy boost: On Fridays and Saturdays during peak hours (12:00-20:00), elevator speed increases by 50% to handle higher traffic

- Time-dependent travel speeds**:** Daytime (07:00-22:00) uses faster travel times (4 seconds per floor), while night-time uses slower speeds (6 seconds per floor) for energy conservation
- Automatic day progression**:** The system simulates time passage with a virtual clock that advances 5 minutes every second
- Seasonal adaptability: Different operational modes can be programmed for different times of day and days of the week

```
handles = struct();
handles.current_floor = 0;
handles.target_floor = 0;
handles.is_moving = false;
handles.direction = 'STOP';
handles.floor_requests = [];  % Request queue
handles.emergency_mode = false;
handles.active_emergency = 'none';
handles.weight_limit_exceeded = false;
```

Graphical User Interface Design

Elevator Status Panel

Continuously displays the current system state:
• Current floor
• Direction (UP, DOWN, IDLE)
• Elevator state (MOVING / STOPPED)
• Door state (OPEN / CLOSED)
• Motion detection status
• Energy-saving mode status
• Emergency status (NONE, FIRE, FLOOD, EARTHQUAKE)

Cabin Interface Panel

Simulates the controls available to passengers inside the elevator. Components include:
• Floor selection buttons: –1, 0, 1, 2, 3
• Door OPEN button
• Door CLOSE button

• Emergency STOP button
• Light ON/OFF button
• Radio (music) ON/OFF button
• Song selection menu
• Volume slider

Control and Monitoring Panel

This panel allows simulation of elevator conditions and safety monitoring:
• Passenger weight input (kg)
• Motion sensor ON/OFF
• Passenger presence indicator
• Manual emergency activation
• Visual alerts and warnings

Emergency Control Panel

This panel represents building-level emergency controls:
• Fire emergency
• Flood emergency
• Earthquake emergency
• System reset

Elevator Movement Simulation

Elevator movement is simulated logically, not through graphical shaft animation.
• Each movement occurs one floor at a time
• Time delays (pause) simulate travel time
• GUI labels update after each step
• Direction changes are displayed textually

Request Handling and Movement Logic

All floor requests are stored in a single queue.
Duplicate requests are automatically ignored to prevent unnecessary stops, and the queue is cleared immediately when any emergency state is activated.
The elevator operates using a continuous control loop:
• The system first checks for an active emergency; if detected, normal operation is suspended.
• If the current floor matches a queued request, the elevator stops, opens the doors,

serves the request, and then closes the doors.
• If no requests are present, the elevator enters an idle state and activates energy-saving mode.
• If requests exist, the system selects the nearest requested floor, moves one floor toward the target, updates the GUI status, and repeats the loop.
This approach provides efficient request servicing while maintaining simple and reliable control logic.

```
%% Nearest-Car Algorithm Implementation
function start_elevator_movement(fig)
    % ... (initial checks for emergency/weight/motion) ...

    % Get all floors in current direction
    if strcmp(handles.direction, 'UP')
        floors_in_direction = handles.floor_requests(handles.floor_requests > handle
s.current_floor);
        if ~isempty(floors_in_direction)
            handles.target_floor = min(floors_in_direction);  % Nearest floor above
        else
            handles.direction = 'DOWN';  % Change direction
            floors_in_direction = handles.floor_requests(handles.floor_requests < han
dles.current_floor);
            handles.target_floor = max(floors_in_direction);  % Nearest floor below
        end
    end
```

Door System and Motion Sensor

Doors open only when the elevator is fully stopped.
A simulated motion sensor prevents door closing when movement is detected and immediately stops the elevator if motion occurs during travel.
Manual door controls override automatic behavior, ensuring passenger safety.

```
%% Safety-Interlocked Door Control
function close_door(src, event, fig)
    if handles.is_moving
        set_status_message('Cannot close doors while elevator is moving', fig);
        return;
    end

    if handles.motion_sensor
        set_status_message('Motion detected - cannot close doors', fig);
        return;
    end
```

Weight Control System

The maximum permitted load is 800 kg.
If this limit is exceeded, the elevator stops, clears all requests, opens the doors, and displays a warning message.
Normal operation resumes only after the load is reduced to a safe level.

```
%% Weight Monitoring with Automatic Safety Response
function update_weight(src, event, fig)
    handles.weight = round(get(handles.weight_slider, 'Value'));

    if handles.weight > 800
        handles.weight_limit_exceeded = true;
        handles.is_moving = false;           % Stop movement
        handles.direction = 'STOP';
        handles.floor_requests = [];         % Clear requests
        handles.door_open = true;            % Open doors
        set_status_message('WARNING: OVERWEIGHT! Maximum 800 kg', fig);
    end
```

Music and Lighting System

Background music plays during normal operation.
Users can select songs and adjust volume.
Music automatically stops during emergencies and when the system enters energy-saving mode.
Lights remain ON during active use and automatically turn OFF during energy-saving

mode.
Manual override is available through the cabin interface.

Emergency Systems

The system includes three emergency modes, each of which overrides all normal elevator operations.
Fire emergency: All active requests are cancelled. The elevator moves automatically to floor 0, opens the doors, displays an alarm message, and remains disabled until a system reset.
Flood emergency: The basement floor (–1) is disabled. The elevator moves to floor 3, opens the doors, and remains disabled until reset.
Earthquake emergency: The elevator stops immediately and all movement is disabled. Doors remain closed, and the system waits for manual reset.

Emergency Priority Logic

Emergency states have absolute priority over all other operations, following this order:

1. Fire, Flood, Earthquake
2. Manual emergency stops
3. Normal operation
   Higher-priority events immediately override lower-priority logic.

```
%% Emergency State Hierarchy
if handles.emergency_mode && ~strcmp(handles.active_emergency, 'none')
    % EMERGENCY PROTOCOL: Cannot accept new floor requests
    set_status_message(sprintf('EMERGENCY ACTIVE (%s)! Cannot accept floor requests',
...
        handles.active_emergency), fig);
    return;
end
```

Energy-Saving Mode

When no passengers or floor requests are detected, the system enters energy-saving mode:
• Lights turn OFF
• Music stops
• Doors close
• Elevator remains idle
• Status displays ENERGY SAVING MODE

```
%% Energy Conservation Logic
function toggle_energy_saving(src, event, fig)
    handles.energy_saving_mode = ~handles.energy_saving_mode;

    if handles.energy_saving_mode
        handles.door_open = false;          % Close doors
        handles.lights_on = false;          % Turn off lights
        handles.radio_playing = false;      % Stop music
        handles.is_moving = false;          % Stop movement
        handles.floor_requests = [];        % Clear requests
        set_status_message('Energy saving mode activated - No passengers', fig);
    end
```

Failure Recovery and Reset

After any emergency event, the system remains locked in a safe state.
Normal operation can only be restored using the RESET button, which returns all variables to their default safe values.

Apparatus and Tools Used

• Laptop / personal computer
• GNU Octave
• MATLAB-style GUI tools
• Online documentation and programming forums

• YouTube educational resources on elevator systems
• Engineering reference materials

QR Code Feedback System

Customer Feedback Integration

To enhance user experience and maintenance efficiency, the system includes a customer feedback mechanism:

- A QR code generator that creates scannable codes linking to feedback forms
- Real-time feedback statistics tracking (compliments, complaints, song requests)
- Automatic maintenance alerts triggered when complaints reach a threshold
- Song request system allowing passengers to suggest music preferences
- The feedback portal opens in a separate window with interactive buttons for different feedback types (complaints, compliments, song requests)

Development Process

• The project was developed in an iterative manner. It started with a simple model of elevator movement, which provided a basic understanding of how the system should operate. The team then researched online examples and explored alternative approaches to broaden their perspective and identify best practices. To gain a deeper understanding, they studied real elevator operations through educational videos, observing how safety and control systems function in practice. Gradually, they added safety features and refined the control logic, ensuring the system would behave reliably under different conditions. Recognizing the importance of a visual interface, they implemented an initial GUI in Octave, allowing users to interact with the simulation and monitor the elevator's state in a clear and intuitive way.
• Refined the system through 11 major revisions (each "revision" involved reviewing the current version, identifying problems or areas for improvement (like bugs, missing features, or design issues), and then modifying the code, GUI, or logic to make the system better, more reliable, or more user-friendly)
• Integrated movement, safety systems, GUI, and energy management
• Performed final testing to ensure stable and realistic behaviour.

Educational Value and Conclusion

This project demonstrates:
• State-based control system design
• Safety-critical decision making
• Human–machine interaction
• GUI-based system simulation
• Practical engineering problem-solving methods

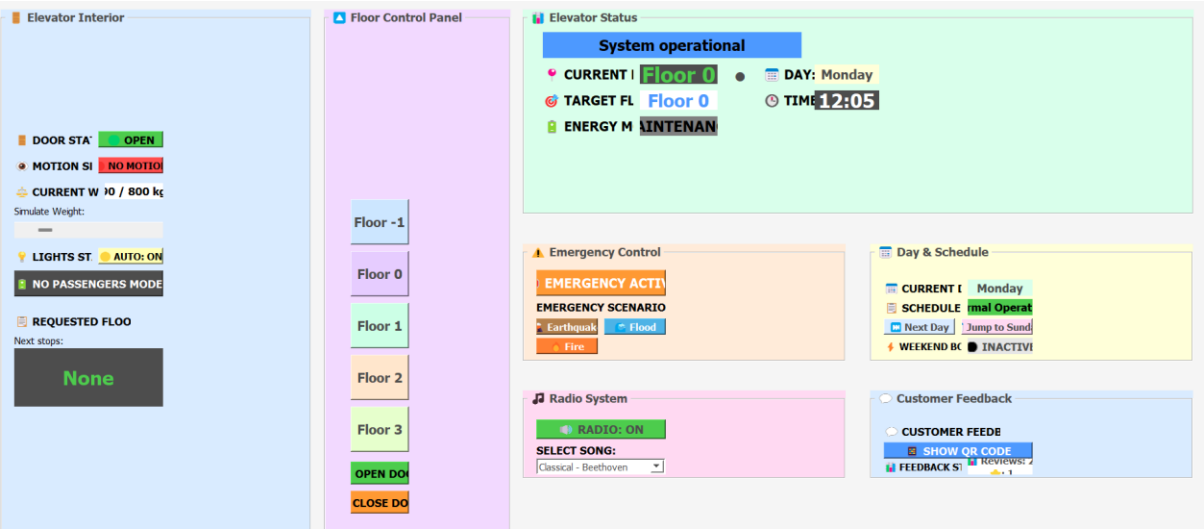
This project provides a comprehensive learning experience by demonstrating the principles of state-based control system design, where each elevator operation—whether idle, moving, or handling emergencies—is modelled as a distinct state with clearly defined transitions. It highlights the importance of safety-critical decision making, showing how the system must prioritize emergencies such as fire, flood, or earthquakes above all other operations to ensure passenger safety.

Through the development of a graphical user interface, the project emphasizes human–machine interaction, allowing users to interact with the system in a way that is both intuitive and informative. The simulation demonstrates how real-time feedback, such as floor status, direction indicators, and emergency alerts, can improve understanding and usability in complex systems.

By combining control logic, safety monitoring, energy management, and user interaction in a single software simulation, the project also illustrates practical engineering problem-solving methods, including iterative design, debugging, and system testing. Students gain insight into how engineers anticipate and address real-world challenges before implementing physical systems, reducing risk and optimizing efficiency.

Overall, the smart five-story elevator simulation effectively shows how complex real-world systems can be modelled, tested, and analysed using software. It provides a realistic platform for exploring the integration of mechanical, electrical, and software components, and it prepares learners to approach engineering challenges with a methodical, safety-conscious, and solution-oriented mindset.

Project interface:



Resources:

- https://codefarm0.medium.com/how-to-design-an-elevator-system-interview-deep-dive-20f6bdb20d37
- https://quantup.ai/intelligent-elevators-transforming-vertical-mobility-with-ai-ml/
- https://youtu.be/jOF-4B-jBUo?si=7D0_SP9ccq8pKoJ_
- https://youdh.co.uk/guest-posts/elevator-simulation-software/
- https://www.tiktok.com/@feleciaforthewin/video/7158164760490560814?q=elevator%20logic&t=1768008489124
- https://www.tiktok.com/@thithuyle974/video/7576998122379922719?q=elevator%20logic&t=1768008489124
- https://github.com/ivochan/elevatorControlSystem
- https://github.com/Darian009/ElevatorControl_MATLAB